Object Oriented Programming Laboratory

Programming Assessment: STL, File Handling and Exception Handling

Language: C++ Syllabus: Units 5 and 6 Difficulty: Moderate

Question 1: Student Academic Performance Management Using STL and File Handling

Problem Statement. Design and develop a C++ program to implement a Student Academic Performance Management System. Student records are stored in a text file in Comma-Separated Values (CSV) format. Each record contains a unique roll number, student name and marks obtained in three subjects. The program should read student records from the file and store them in an appropriate STL container for further processing.

Sample input file: students.txt

```
101,Akash,75,82,68
102,Priya,88,91,85
103,Rahul,62,70,65
104,Neha,92,89,95
105,Amit,55,48,60
106,Sneha,78,84,80
```

Functional requirements

1. Load Student Records: Read the input file and store the records using an STL vector.

2. Display Student Records: Display roll number, name, subject marks, total and percentage.

3. Sort Students: Display records in descending order of percentage using STL sort().

4. Search Student: Search by roll number and display the corresponding record.

5. Identify Top Performer: Find the student with the highest percentage using an appropriate STL algorithm.

6. Generate Result File: Create results.txt containing roll number, name, total, percentage and Pass/Fail status. A student passes only when each subject mark is at least 40.

7. Exit: Terminate the program.

Exception handling requirements

• Input file does not exist or cannot be opened.

• A student record contains missing or invalid fields.

• Subject marks fall outside the range 0-100.

• Duplicate roll numbers are present.

• An entered roll number does not exist.

Use appropriate try, throw and catch statements to display meaningful error messages. Report and skip invalid individual records without preventing valid records from loading.

Concepts: CSV file handling, STL vector, sort(), find_if(), max_element(), exception handling.

Question 2: Service Request Scheduling System Using STL Priority Queue and File Handling

Problem Statement. Design and develop a C++ program to implement a Service Request Scheduling and Processing System for a technical support center. Requests are stored in a CSV text file containing a unique request ID, customer name and priority level. Read all service requests and process them using STL containers.

Sample input file: requests.txt

```
101,Akash,Low
102,Priya,High
103,Rahul,Medium
104,Neha,High
105,Amit,Low
106,Sneha,Medium
107,Rohan,High
108,Pooja,Medium
```

Scheduling rules

• Priority order: High before Medium before Low.

• FIFO within equal priority: Requests of equal priority are processed in their order of appearance in the input file.

• Removal: A processed request is removed from the pending queue.

Functional requirements

1. Load Service Requests: Read CSV records into an STL priority_queue using an appropriate comparator.

2. Display Pending Requests: Show pending requests in expected processing order.

3. Process Next Request: Process the highest-priority pending request; break ties by arrival order.

4. Process All Requests: Process all remaining requests using the scheduling rules.

5. Search Request: Search for a pending request by ID.

6. Generate Processing Log: Write each processed request to processed_requests.txt in actual processing order.

7. Display Statistics: Show the processed-request count and pending counts for each priority.

8. Exit: Terminate the program.

Expected processing order for sample data

Order	Request ID	Customer	Priority
1	102	Priya	High
2	104	Neha	High
3	107	Rohan	High
4	103	Rahul	Medium
5	106	Sneha	Medium
6	108	Pooja	Medium

Order	Request ID	Customer	Priority
7	101	Akash	Low
8	105	Amit	Low

Exception handling requirements

• Input file cannot be opened.

• A request has an invalid priority.

• A duplicate request ID is present.

• A record has missing or invalid fields.

• The user attempts processing when the queue is empty.

• The processing log cannot be opened or written.

Use appropriate exception handling. Report and skip invalid individual input records without terminating the program unexpectedly.

Concepts: STL priority_queue, custom comparator, FIFO scheduling, CSV file handling, exception handling.

Common Implementation Instructions

1. Use appropriate C++ classes or structures to represent records.

2. Provide a menu using a while or do-while loop and switch statement.

3. Continue executing until Exit is selected.

4. Use STL containers and algorithms instead of manually implementing equivalent structures.

5. Read initial records from external text files.

6. Use try, throw and catch to demonstrate exception handling.

7. Validate input and display informative error messages.

Suggested Evaluation Scheme (25 marks per program)

Evaluation criterion	Marks
Class/structure design and data representation	3
File reading and CSV parsing	5
Correct use of STL containers and algorithms	6
Implementation of menu operations	5
Exception handling and input validation	4
Program readability and execution	2
TOTAL	**25**

Question 1 emphasizes vector and STL algorithms. Question 2 emphasizes priority_queue, custom comparators and scheduling logic. Both integrate file handling and exception handling.